Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen IX: Ingeniería del Software, Arquitectura, Sistemas Distribuidos, Seguridad y Proyectos

Enciclopedia de Programación e Ingeniería del Software — Partes IX, X, XI, XII y XIII

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Partes IX–XIII: Ingeniería Profesional, Cloud, Seguridad e Itinerarios 1

1. Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización 2

1.1. Cohesión, Acoplamiento y Métrica de Inestabilidad de Paquetes y Los Cinco Principios SOLID bajo una Lente Moderna . . . 2

1.2. Patrones de Diseño GoF en Lenguajes Modernos: De Clases a Funciones y Tipos Algebraicos y Deuda Técnica, Code Smells y Disciplina de Refactorización Segura . . . 2

2. Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC) 5

2.1. Domain-Driven Design (DDD): Diseño Estratégico y Táctico y Arquitectura Hexagonal (Puertos y Adaptadores) y Arquitectura Limpia . . . 5

2.2. Monolito Modular frente a Microservicios, CQRS y Patrón Outbox Transaccional y Diseño de APIs: REST (OpenAPI), GraphQL, gRPC y WebSockets . . . 6

3. Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing 8

3.1. Arquitectura Interna de Git: Almacén Direccionable por Contenido y Grafo DAG y Pirámide de Testing, TDD y Dobles de Prueba (Stubs, Fakes, Mocks) . . . 8

3.2. Property-Based Testing (QuickCheck, Hypothesis, proptest) y Reducción Automática (Shrinking) y Fuzzing Guiado por Cobertura (libFuzzer, AFL++) y Mutation Testing . . . 8

4. Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE 11

4.1. Tiempo Físico vs Tiempo Lógico: Relojes de Lamport y Relojes Vectoriales y Consenso Distribuido con Raft: Elección de Líder y Replicación de Log . . . 11

4.2. Patrones de Resiliencia: Timeouts, Exponential Backoff con Jitter, Circuit Breaker e Idempotencia y Observabilidad con OpenTelemetry (Métricas, Logs, Trazas) y Disciplina SRE (SLI/SLO) . . . 11

5. Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain 14

5.1. Modelado de Amenazas (STRIDE) y Defensa en Profundidad ante OWASP Top 10 y Criptografía Aplicada Moderna: TLS 1.3, AEAD (AES-GCM / ChaCha20-Poly1305) y Argon2id . . . 14

5.2. Autenticación y Autorización Moderna: OAuth 2.1, OpenID Connect (OIDC), PKCE y Passkeys y Seguridad de la Cadena de Suministro (Supply Chain): Lockfiles, SBOM, Sigstore y SLSA . . . 15

6. Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD 17

6.1. Anatomía de los Contenedores Linux: Namespaces, cgroups v2 y OverlayFS y Orquestación con Kubernetes: Bucle de Reconciliación, Pods, Probes y Autoscaling . . . 17

6.2. Infraestructura como Código (IaC) con Terraform y Gestión de Estado y Pipelines CI/CD y Estrategias de Despliegue Progresivo (Blue/Green, Canary, Feature Flags) . . . 18

7. Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados **20**
7.1. Arquitectura de los 19 Itinerarios de Aprendizaje Personalizados y Proyectos Integradores de Sistemas: Motor KV en Rust, Arena Allocator en C17 y Planificador en C++20 20
7.2. Proyectos Integradores de Alto Nivel y Datos: VM en Python 3.12, Router HTTP en TypeScript y Ledger SQL y Rúbricas de Evaluación por Niveles de Competencia (Principiante a Experto) 21

A. Notación **23**

B. Hoja de fórmulas **24**

C. Soluciones **25**
C.1. Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización . 25
C.2. Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC) 25
C.3. Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing 26
C.4. Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE 27
C.5. Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain 27
C.6. Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD 28
C.7. Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados 28

D. Auditoría de fuentes **30**

E. Plan de estudio **32**

Parte I

Partes IX–XIII: Ingeniería Profesional, Cloud, Seguridad e Itinerarios

Capítulo 1

Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización

Fundamentos del diseño modular de software mantenible: cohesión funcional, inversión de dependencias y catálogo crítico de patrones creacionales, estructurales y de comportamiento.

Consejo

Muchos de los 23 patrones clásicos de Gamma, Helm, Johnson y Vlissides (1994) se simplifican radicalmente mediante funciones de orden superior, tipos suma y traits.

Definición

Cohesión, Acoplamiento y Métrica de Inestabilidad de Paquetes: El diseño modular busca maximizar la cohesión interna de cada módulo mientras dirige el acoplamiento hacia abstracciones estables.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La refactorización es una transformación del código fuente que preserva estrictamente el comportamiento externo observable mientras mejora su estructura interna.

1.1 Cohesión, Acoplamiento y Métrica de Inestabilidad de Paquetes y Los Cinco Principios SOLID bajo una Lente Moderna

La complejidad accidental de un sistema crece cuadráticamente con el acoplamiento implícito entre sus componentes. Dado un módulo o paquete, su acoplamiento aferente C_a cuenta cuántos módulos externos dependen de él, y su acoplamiento eferente C_e cuenta de cuántos módulos externos depende él. La métrica de inestabilidad $I = C_e/(C_a + C_e) \in [0, 1]$ caracteriza desde paquetes máximamente estables ($I = 0$, como el modelo de dominio puro) hasta paquetes volátiles ($I = 1$, como adaptadores de interfaz o puntos de entrada `main`). El Principio de Dependencias Estables dicta que un módulo sólo debe depender de módulos con menor o igual inestabilidad. 1) **Responsabilidad Única (SRP)**: un módulo debe tener una única razón de cambio asociada a un actor de negocio. 2) **Abierto/Cerrado (OCP)**: el comportamiento debe poder extenderse mediante polimorfismo o composición de funciones sin modificar el código ya verificado. 3) **Sustitución de Liskov (LSP)**: formalizado por Barbara Liskov y Jeannette Wing, exige que todo subtipo preserve los invariantes e historia observable del supertipo. 4) **Segregación de Interfaces (ISP)**: ningún cliente debe verse forzado a depender de métodos que no utiliza. 5) **Inversión de Dependencias (DIP)**: los módulos de alto nivel definen las interfaces (traits, protocolos) que implementan los módulos de infraestructura de bajo nivel.

1.2 Patrones de Diseño GoF en Lenguajes Modernos: De Clases a Funciones y Tipos Algebraicos y Deuda Técnica, Code Smells y Disciplina de Refactorización Segura

Los patrones creacionales (`Factory Method`, `Abstract Factory`, `Builder`) encapsulan la construcción válida de objetos complejos; en particular, el patrón `Type-State Builder` en Rust o TypeScript verifica en tiempo de compilación que todos los campos obligatorios se hayan configurado antes de invocar

`.build()`. Los patrones estructurales (`Adapter`, `Decorator`, `Facade`, `Proxy`, `Composite`) traducen interfaces o añaden responsabilidades transversales (telemetría, reintentos, caché) sin alterar la implementación subyacente. Los patrones de comportamiento (`Strategy`, `Observer`, `Command`, `State`, `Iterator`) modelan algoritmos intercambiables y máquinas de estados. Acuñada por Ward Cunningham, la metáfora de la deuda técnica describe el interés compuesto que se paga en cada entrega futura cuando se pospone el reflejo de un aprendizaje de dominio en la arquitectura del código. Toda refactorización (extracción de funciones puras, sustitución de condicionales anidados por polimorfismo o pattern matching exhaustivo, introducción de objetos valor) debe ejecutarse en pasos atómicos respaldados por una batería rápida de pruebas automatizadas.

Problema resuelto

Enunciado En una arquitectura modular, el paquete de facturación es importado por 18 módulos externos (acoplamiento aferente Ca = 18) e importa únicamente 2 bibliotecas estándar de tipos (acoplamiento eferente Ce = 2). Calcule el índice de inestabilidad I expresado en porcentaje.
Datos

- $C_a = 18\,\text{módulos}$ — Acoplamiento aferente (dependientes entrantes)
- $C_e = 2\,\text{módulos}$ — Acoplamiento eferente (dependencias salientes)

Incógnita Índice de inestabilidad porcentual I_pct en %
Modelo físico
Cálculo de la métrica de inestabilidad de un paquete de dominio

$$I_{\text{pct}} = \frac{2}{18+2} \cdot 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Aplicamos la fórmula de Martin I = Ce / (Ca + Ce) multiplicada por 100

$$I_{\text{pct}} = \frac{2}{18+2} \cdot 100$$

$$\frac{(2 \cdot 100)}{(18+2)}$$

Resultado

$$\textit{Resultado} = 10$$

Interpretación El paquete presenta una inestabilidad baja del 10 %, acorde con su rol como núcleo de negocio estable del que dependen otros 18 módulos.
Comprobación de los pasos

- Paso 1: 10 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) [CH-01.1] Calcule (Índice de inestabilidad porcentual de un adaptador HTTP con Ca = 1 y Ce = 9) expresado en %.
2 [PRB-0004] (Intermedio, Desarrollo por pasos) [CH-01.2] Calcule (Distancia a la secuencia principal en porcentaje si A = 40 % e I = 50 % (|40 + 50 - 100|)) expresado en %.

3 [PRB-0005] (Intermedio, Desarrollo por pasos) [CH-01.3] Calcule (Reducción de complejidad ciclomática al descomponer un método de complejidad 19 en 4 funciones de complejidad 2) expresado en puntos.
4 [PRB-0006] (Avanzado, Diseño) [CH-01.4] Calcule (Número de clases eliminadas al reemplazar 6 clases Strategy de un solo método por funciones de primera clase) expresado en clases.

Capítulo 2

Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC)

Arquitectura de sistemas de software a escala empresarial: fronteras de dominio, estilos arquitectónicos, CQRS, Event Sourcing y diseño de contratos de comunicación.

Consejo

Un monolito modular bien acotado ofrece fronteras lógicas limpias con transacciones ACID locales y cero sobrecoste de red, debiendo extraerse microservicios solo ante necesidades reales de escalado o despliegue independiente.

Definición

Domain-Driven Design (DDD): Diseño Estratégico y Táctico: El diseño estratégico divide un dominio complejo en Contextos Acotados (Bounded Contexts) con Lenguaje Ubicuo propio, evitando el antipatrón de un modelo canónico único gigante.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: REST sobre HTTP/JSON sobresale en interoperabilidad pública, GraphQL optimiza consultas de clientes con grafos de vista diversos y gRPC sobre HTTP/2 + Protobuf domina la comunicación interna de baja latencia.

2.1 Domain-Driven Design (DDD): Diseño Estratégico y Táctico y Arquitectura Hexagonal (Puertos y Adaptadores) y Arquitectura Limpia

En el nivel estratégico, DDD identifica subdominios principales (Core, donde reside la ventaja competitiva), de soporte y genéricos, estableciendo mapas de contexto (Context Maps) con capas anticorrupción (ACL) para traducir conceptos entre sistemas. En el nivel táctico, distingue **Entidades** (definidas por su identidad persistente a lo largo del tiempo), **Objetos Valor** (definidos exclusivamente por sus atributos e inmutables por construcción, como `ImporteMonetario`), **Agregados** (fronteras de consistencia transaccional gobernadas por su Aggregate Root) y **Eventos de Dominio** (hechos inmutables ocurridos en el pasado). Formulada por Alistair Cockburn y popularizada junto a la Arquitectura Limpia de Robert C. Martin, la Arquitectura Hexagonal representada en la figura 1 organiza el software en anillos concéntricos. El núcleo contiene las entidades y casos de uso de aplicación; alrededor suyo se sitúan los puertos de entrada (invocados por controladores HTTP, consumidores de colas o CLI) y los puertos de salida (interfaces de repositorios, pasarelas de pago o relojes de sistema), cuyas implementaciones concretas residen en la capa externa de infraestructura.

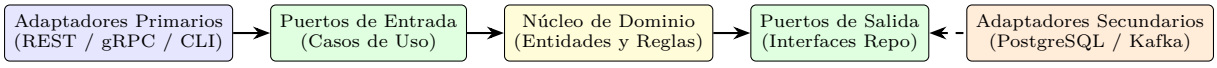


Figura 1. Arquitectura Hexagonal (Puertos y Adaptadores): los adaptadores primarios invocan puertos de entrada (casos de uso) y el dominio invoca puertos de salida implementados por adaptadores secundarios.

2.2 Monolito Modular frente a Microservicios, CQRS y Patrón Outbox Transaccional y Diseño de APIs: REST (OpenAPI), GraphQL, gRPC y WebSockets

Distribuir prematuramente un sistema sin fronteras de dominio maduras engendra un 'monolito distribuido' que sufre la latencia, los fallos parciales y la complejidad operativa de los microservicios sin obtener autonomía de despliegue. Cuando un sistema requiere publicar eventos en un broker (Kafka, RabbitMQ) tras actualizar su base de datos, escribir simultáneamente en ambos sin coordinación dos fases es inseguro; el patrón **Transactional Outbox** resuelve este problema insertando el evento en una tabla `outbox` dentro de la misma transacción ACID local y reenviándolo de forma asíncrona con garantía `at-least-once`. En una API RESTful madura, los recursos se modelan como sustantivos jerárquicos, los métodos HTTP respetan su semántica de seguridad e idempotencia (`GET` y `HEAD` seguros; `PUT` y `DELETE` idempotentes; `POST` con clave `Idempotency-Key` para pagos), y los errores se devuelven en formato estándar RFC 9457 (`Problem Details for HTTP APIs`). En GraphQL es imprescindible implementar el patrón `DataLoader` para agrupar consultas y evitar el problema $N + 1$.

Problema resuelto

Enunciado Una consulta GraphQL solicita una lista de 50 pedidos junto con los datos del autor de cada pedido. Sin agrupación (N+1), el servidor ejecuta 1 consulta para la lista y 50 consultas individuales de 2 ms de latencia cada una. Con DataLoader, ejecuta solo 2 consultas (la lista y un único SELECT ... WHERE id IN (...)) de 2 ms cada una. Calcule el tiempo total de base de datos ahorrado en ms.
Datos

- $N = 50\,\text{pedidos}$ — Número de elementos de la lista
- $t_q = 2\,\text{ms}$ — Latencia por consulta a la base de datos

Incógnita Tiempo de base de datos ahorrado T_ahorro en ms
Modelo físico
Eliminación del problema N+1 en un resolver GraphQL mediante DataLoader

$$T_{\text{ahorro}} = (50 + 1) \cdot 2 - 2 \cdot 2$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el tiempo de las 2 consultas agrupadas al tiempo de las N + 1 = 51 consultas secuenciales

$$T_{\text{ahorro}} = (50 + 1) \cdot 2 - 2 \cdot 2$$

$$(((50 + 1) \cdot 2) - (2 \cdot 2))$$

Resultado

$$Resultado = 98$$

Interpretación DataLoader reduce el tiempo de consultas de 102 ms a 4 ms, ahorrando exactamente 98 ms por petición.
Comprobación de los pasos

- Paso 1: 98 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) [CH-02.1] Calcule (Consultas SQL ahorradas al resolver 120 elementos con DataLoader (2 consultas) frente a N+1 (121 consultas)) expresado en consultas.
2 [PRB-0010] (Intermedio, Desarrollo por pasos) [CH-02.2] Calcule (Latencia acumulada en ms de una cadena síncrona de 6 microservicios con 8 ms por salto) expresado en ms.
3 [PRB-0011] (Intermedio, Desarrollo por pasos) [CH-02.3] Calcule (Disponibilidad porcentual aproximada (en centésimas x100) de 3 servicios en serie al 99 % cada uno (10000 - 3*100)) expresado en centésimas de %.
4 [PRB-0012] (Avanzado, Diseño) [CH-02.4] Calcule (Tamaño de página máximo devuelto si una API REST limita $page_size <= 100$ cuando el cliente solicita 5000) expresado en registros.

Capítulo 3

Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing

Ingeniería de calidad y verificación de software: modelo de objetos de Git, TDD, dobles de prueba, pruebas basadas en propiedades y análisis de mutación.

Consejo

El Property-Based Testing genera cientos de entradas aleatorias estructuradas para refutar invariantes universales y reduce automáticamente el contraejemplo al caso mínimo reproducible.

Definición

Arquitectura Interna de Git: Almacén Direccionable por Contenido y Grafo DAG: Git no almacena diferencias incrementales por commit, sino instantáneas completas modeladas como un grafo acíclico dirigido (DAG) de cuatro objetos inmutables: blob, tree, commit y tag.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La cobertura de líneas del 100 % no garantiza que las aserciones detecten defectos; el Mutation Testing mide la eficacia real de los tests inyectando mutantes sintácticos.

3.1 Arquitectura Interna de Git: Almacén Direccionable por Contenido y Grafo DAG y Pirámide de Testing, TDD y Dobles de Prueba (Stubs, Fakes, Mocks)

Bajo el directorio `.git/objects`, Git opera como un sistema de archivos clave-valor direccionable por contenido (Content-Addressable Storage) donde la clave de cada objeto es el hash criptográfico (SHA-1 o SHA-256) de su cabecera más su contenido comprimido con zlib. Un **blob** almacena los bytes de un archivo sin su nombre; un **tree** asocia nombres de archivo y permisos con hashes de blobs y subtrees; y un **commit** apunta al tree raíz de la instantánea, a sus commits padres y a los metadatos de autoría. Las ramas (`refs/heads/*`) son simples punteros móviles de 41 bytes hacia un commit. El ciclo Test-Driven Development (TDD: Red -> Green -> Refactor) utiliza las pruebas como herramienta de diseño de interfaces antes de escribir la implementación. Siguiendo la taxonomía de Gerard Meszaros, es fundamental distinguir entre **Stubs** (devuelven respuestas preconfiguradas), **Fakes** (implementaciones funcionales ligeras, como un repositorio en memoria) y **Mocks** (verifican interacciones salientes). Abusar de mocks acoplados a detalles internos produce pruebas frágiles que se rompen ante cualquier refactorización legítima.

3.2 Property-Based Testing (QuickCheck, Hypothesis, proptest) y Reducción Automática (Shrinking) y Fuzzing Guiado por Cobertura (libFuzzer, AFL++) y Mutation Testing

Mientras una prueba basada en ejemplos comprueba una tupla concreta (`assert ordenar([3,1,2]) == [1,2,3]`), una prueba basada en propiedades (introducida por Claessen y Hughes en QuickCheck para Haskell y disponible en Hypothesis para Python o `proptest` para Rust) declara propiedades algebraicas que deben cumplirse para toda entrada válida: idempotencia (`f(f(x)) == f(x)`), inversión

(`decode(encode(x)) == x`) o preservación de invariantes estructurales. Al encontrar un fallo, el motor aplica *shrinking* en el árbol de generación para entregar el contraejemplo más simple. En analizadores de protocolos, compiladores y parsers binarios, el Fuzzing guiado por cobertura (Coverage-Guided Fuzzing con `cargo-fuzz` / `libFuzzer`) instrumenta las aristas del grafo de control de flujo y muta el corpus de entradas cada vez que una mutación descubre una nueva ruta de ejecución. Por su parte, herramientas de Mutation Testing (`mutmut`, `cargo-mutants`, `Stryker`) modifican operadores en el código de producción (`<` por `<=`, `+` por `-`) y verifican qué porcentaje de mutantes son detectados ('muertos') por la suite de tests.

Problema resuelto

Enunciado Una herramienta de Mutation Testing genera 250 mutantes sintácticos sobre el módulo de liquidación financiera. De ellos, 10 son mutantes equivalentes (no alteran la semántica) y la batería de pruebas detecta y hace fallar 216 de los mutantes restantes. Calcule el Mutation Score porcentual sobre los mutantes no equivalentes.

Datos

- $M_{tot} = 250\,\text{mutantes}$ — Total de mutantes generados
- $M_{eq} = 10\,\text{mutantes}$ — Mutantes semánticamente equivalentes
- $M_{kill} = 216\,\text{mutantes}$ — Mutantes detectados (killed) por los tests

Incógnita Puntuación de mutación S_mut en %

Modelo físico

Cálculo del Mutation Score en una auditoría de calidad de pruebas

$$S_{\text{mut}} = \frac{216}{250 - 10} \cdot 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos los 216 mutantes detectados entre los 240 mutantes no equivalentes (250 - 10) y multiplicamos por 100

$$S_{\text{mut}} = \frac{216}{250 - 10} \cdot 100$$

$$\frac{(216 \cdot 100)}{(250 - 10)}$$

Resultado

$$\textit{Resultado} = 90$$

Interpretación La suite de pruebas alcanza un Mutation Score del 90 %, identificando exactamente los 24 mutantes supervivientes que requieren nuevas aserciones.

Comprobación de los pasos

- Paso 1: 90 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) [CH-03.1] Calcule (Mutantes supervivientes que requieren nuevos tests en el módulo auditado (250 - 10 - 216)) expresado en mutantes.

2 [PRB-0016] (Intermedio, Desarrollo por pasos) [CH-03.2] Calcule (Objetos nuevos creados en Git al modificar 1 archivo dentro de un subdirectorio de profundidad 2 (1 blob + 3 trees + 1 commit)) expresado en objetos.
3 [PRB-0017] (Intermedio, Desarrollo por pasos) [CH-03.3] Calcule (Casos generados en 5 segundos por un fuzzer libFuzzer que ejecuta 18 000 iteraciones por segundo) expresado en ejecuciones.
4 [PRB-0018] (Avanzado, Diseño) [CH-03.4] Calcule (Tiempo total de CI en segundos para 1 200 tests unitarios de 2 ms y 30 tests de integración de 200 ms) expresado en s.

Capítulo 4

Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE

Teoría y práctica de los sistemas distribuidos modernos: falacias de la red, relojes de Lamport/vectoriales, consenso Raft, trazabilidad distribuida e ingeniería de fiabilidad.

Consejo

Reintentar llamadas fallidas inmediatamente o con intervalos fijos provoca tormentas de reintentos (Thundering Herd) que tumban al servicio en recuperación; añadir Jitter aleatorio descorrela la carga.

Definición

Tiempo Físico vs Tiempo Lógico: Relojes de Lamport y Relojes Vectoriales: En ausencia de un reloj físico global perfectamente sincronizado, la relación happened-before de Lamport define un orden parcial estricto sobre los eventos de un sistema distribuido.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: El contexto de propagación W3C Trace Context (traceparent) correlaciona logs y métricas a través de decenas de servicios mediante un identificador global de traza y spans jerárquicos.

4.1 Tiempo Físico vs Tiempo Lógico: Relojes de Lamport y Relojes Vectoriales y Consenso Distribuido con Raft: Elección de Líder y Replicación de Log

Los relojes físicos de cuarzo en servidores distintos sufren deriva térmica, y la sincronización NTP presenta incertidumbres de milisegundos que impiden ordenar causalmente eventos de microsegundos. Leslie Lamport formalizó en 1978 la relación causal $a \to b$ (happened-before): si $a \to b$, entonces las marcas lógicas cumplen $L(a) < L(b)$. Los **Relojes Vectoriales** $V \in \mathbb{N}^N$ extienden esta propiedad logrando una equivalencia exacta: $V(a) < V(b) \iff a \to b$, permitiendo detectar exactamente cuándo dos escrituras son concurrentes y requieren resolución de conflictos (CRDTs o reconciliación). En Raft, el tiempo se divide en mandatos (*terms*) numerados monotónicamente. Todo nodo se encuentra en uno de tres estados: *Follower*, *Candidate* o *Leader*. Si un seguidor no recibe latidos (`AppendEntries` vacíos) del líder durante su tiempo de espera aleatorio (p. ej., $150 - 300$ ms), incrementa su mandato, pasa a candidato y solicita votos (`RequestVote`). La restricción de votación garantiza que sólo un candidato cuyo log esté al menos tan actualizado como el de la mayoría de los nodos puede ser elegido líder.

4.2 Patrones de Resiliencia: Timeouts, Exponential Backoff con Jitter, Circuit Breaker e Idempotencia y Observabilidad con OpenTelemetry (Métricas, Logs, Trazas) y Disciplina SRE (SLI/SLO)

Toda llamada de red debe poseer un `timeout` explícito y acotado. Cuando una operación es idempotente (o transporta un token único `Idempotency-Key` registrado atómicamente por el receptor), los fallos transitorios pueden reintentarse aplicando espera exponencial con jitter completo: $t_{\text{wait}} \sim U(0, \text{mín}(T_{\text{máx}}, T_{\text{base}} \cdot$

2^{intento})). Si la tasa de fallos supera un umbral crítico, un **Circuit Breaker** abre el circuito y falla rápidamente en local para permitir que el sistema aguas abajo se recupere. OpenTelemetry estandariza la instrumentación de telemetría neutral respecto al proveedor: **Métricas** (contadores, gauges e histogramas de latencia p50/p95/p99), **Logs estructurados** en JSON y **Trazas distribuidas** (grafos de *spans* con `trace_id` y `span_id`). En Site Reliability Engineering (SRE), la fiabilidad se gobierna mediante Indicadores de Nivel de Servicio (SLI, p. ej., proporción de peticiones HTTP exitosas en menos de 200 ms) y Objetivos (SLO, p. ej., 99,9 % mensual, lo que otorga un presupuesto de error de 43,2 minutos cada 30 días).

Problema resuelto

Enunciado Un servicio crítico de pagos define un SLO de disponibilidad del 99,9 % medido sobre una ventana de 30 días (43 200 minutos totales). Calcule el presupuesto de error mensual en minutos durante los cuales el servicio puede experimentar indisponibilidad sin violar su SLO.
Datos

- $T_{mes} = 43200\,\text{min}$ — Minutos totales en 30 días (30 x 24 x 60)
- $p_{err} = 0{,}001\,\text{fracción}$ — Fracción permitida de error (1 - 0,999)

Incógnita Presupuesto de error E_min en minutos
Modelo físico
Cálculo del Presupuesto de Error (Error Budget) mensual para un SLO del 99,9 %

$$E_{\text{min}} = \frac{43200}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 43 200 minutos del mes por la fracción permitida de indisponibilidad (1 / 1000)

$$E_{\text{min}} = \frac{43200}{1000}$$

$$\frac{43200}{1000}$$

Resultado

$$Resultado = 43{,}2$$

Interpretación Un SLO de 'tres nueves' (99,9 %) concede exactamente 43,2 minutos de presupuesto de error cada 30 días.
Comprobación de los pasos

- Paso 1: 43,2 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) [CH-04.1] Calcule (Presupuesto de error anual en minutos para un SLO de cuatro nueves 99,99 % (525600 / 10000)) expresado en min.
2 [PRB-0022] (Intermedio, Desarrollo por pasos) [CH-04.2] Calcule (Nodos totales mínimos en un clúster Raft para tolerar f = 3 fallos simultáneos (2 * 3 + 1)) expresado en nodos.

3 [PRB-0023] (Intermedio, Desarrollo por pasos) [CH-04.3] Calcule (Tope de espera en ms en el cuarto reintento (intento=4) con backoff exponencial de base 50 ms ($50 * 2^4$)) expresado en ms.
4 [PRB-0024] (Avanzado, Diseño) [CH-04.4] Calcule (Disponibilidad porcentual de un servicio con MTTF = 990 horas y MTTR = 10 horas (990 * 100 / (990 + 10))) expresado en %.

Capítulo 5

Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain

Ingeniería de seguridad defensiva desde el diseño (Secure by Design): modelado de amenazas STRIDE, criptografía moderna, identidad federada y seguridad de dependencias.

Consejo

El flujo Authorization Code con PKCE (RFC 7636) protege tanto a aplicaciones móviles y SPA como a servidores web frente a la interceptación del código de autorización.

Definición

Modelado de Amenazas (STRIDE) y Defensa en Profundidad ante OWASP Top 10: La seguridad no es una capa añadida al final del desarrollo, sino una propiedad emergente del diseño verificada mediante el principio de mínimo privilegio y fronteras de confianza explícitas.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Fijar hashes criptográficos en archivos de bloqueo (Cargo.lock, package-lock.json), generar inventarios SBOM y firmar artefactos en CI/CD protege contra ataques de confusión de dependencias.

5.1 Modelado de Amenazas (STRIDE) y Defensa en Profundidad ante OWASP Top 10 y Criptografía Aplicada Moderna: TLS 1.3, AEAD (AES-GCM / ChaCha20-Poly1305) y Argon2id

La metodología STRIDE clasifica las amenazas sobre cada flujo de datos en seis categorías: **S**poofing (suplantación de identidad), **T**ampering (manipulación de datos), **R**epudiation (repudio sin rastro auditable), **I**nformation Disclosure (fuga de información), **D**enial of Service (denegación de servicio) y **E**levation of Privilege (escalada de privilegios). Para neutralizar las vulnerabilidades más críticas del OWASP Top 10—como Broken Access Control (IDOR), Injection (SQLi, XSS, Command Injection) y Server-Side Request Forgery (SSRF)— el sistema debe verificar la autorización a nivel de objeto en cada petición, utilizar consultas parametrizadas, aplicar cabeceras `Content-Security-Policy` (CSP) y restringir las conexiones salientes del servidor hacia rangos IP privados o de metadatos cloud (`169.254.169.254`). Las comunicaciones en tránsito deben protegerse con **TLS 1.3** (RFC 8446), que completa el handshake en 1-RTT y exige intercambio de claves efímeras Diffie-Hellman sobre curvas elípticas (ECDHE con X25519) para garantizar Secreto Perfecto hacia Adelante (*Forward Secrecy*). Para datos en reposo, los esquemas **AEAD** (Authenticated Encryption with Associated Data) como `AES-256-GCM` o `ChaCha20-Poly1305` proporcionan simultáneamente confidencialidad e integridad mediante una etiqueta de autenticación de 128 bits. Para contraseñas, **Argon2id** exige una cantidad configurable de memoria RAM (p. ej., 64 MiB por hash) que neutraliza ataques paralelos con GPUs y ASICs.

5.2 Autenticación y Autorización Moderna: OAuth 2.1, OpenID Connect (OIDC), PKCE y Passkeys y Seguridad de la Cadena de Suministro (Supply Chain): Lockfiles, SBOM, Sigstore y SLSA

OAuth 2.1 delega autorización mientras que OpenID Connect (OIDC) añade una capa de identidad mediante un `ID Token` firmado en formato JWT (con algoritmos asimétricos `RS256` o `EdDSA`, verificando siempre la cabecera `alg`, el emisor `iss`, la audiencia `aud` y la expiración `exp`). En el flujo con **PKCE** (Proof Key for Code Exchange), el cliente genera un secreto efímero `code_verifier` y envía su hash SHA-256 (`code_challenge`) al iniciar el login, impidiendo que un atacante que intercepte el código de retorno pueda canjearlo por tokens. El software moderno depende de cientos de paquetes transitivos de código abierto. Asegurar la cadena de suministro conforme al marco **SLSA** (Supply-chain Levels for Software Artifacts) requiere compilar en entornos efímeros reproducibles, verificar sumas SHA-256 en lockfiles, emitir una lista de materiales de software (**SBOM** en formato CycloneDX o SPDX) y firmar imágenes y binarios sin claves de larga duración mediante certificados efímeros de **Sigstore/Cosign**.

Problema resuelto

Enunciado Un servidor configura Argon2id con un coste de memoria de 64 MiB por verificación de contraseña y 3 iteraciones. Si un atacante intenta evaluar en paralelo 512 candidatos simultáneos en una GPU, calcule la memoria VRAM total en GiB (1 GiB = 1024 MiB) necesaria para sostener esos 512 hilos sin penalización de ancho de banda.

Datos

- $M_{hash} = 64\,\text{MiB}$ — Memoria requerida por instancia de Argon2id
- $N_{par} = 512\,\text{hilos}$ — Candidatos evaluados en paralelo
- $K = 1024\,\text{MiB}\,\text{GiB}^{-1}$ — Mebibytes por gibibyte

Incógnita Memoria VRAM requerida V_gib en GiB

Modelo físico

Coste de memoria agregado de Argon2id frente a ataques de fuerza bruta en GPU

$$V_{\text{gib}} = \frac{512 \cdot 64}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 512 hilos por 64 MiB y dividimos entre 1024 MiB/GiB

$$V_{\text{gib}} = \frac{512 \cdot 64}{1024}$$

$$\frac{(512 \cdot 64)}{1024}$$

Resultado

$$\mathit{Resultado} = 32$$

Interpretación Evaluar tan solo 512 contraseñas en paralelo exige 32 GiB de VRAM dedicada, reduciendo en varios órdenes de magnitud la velocidad de fuerza bruta respecto a SHA-256.

Comprobación de los pasos

- Paso 1: 32 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) [CH-05.1] Calcule (Bytes totales de sobrecoste criptográfico en AES-256-GCM por mensaje (12 bytes de nonce IV + 16 bytes de tag)) expresado en bytes.
2 [PRB-0028] (Intermedio, Desarrollo por pasos) [CH-05.2] Calcule (Bits de entropía de una clave aleatoria de 32 bytes generada con un CSPRNG del sistema operativo (32 * 8)) expresado en bits.
3 [PRB-0029] (Intermedio, Desarrollo por pasos) [CH-05.3] Calcule (Reducción de latencia de establecimiento en ms al pasar de TLS 1.2 (2-RTT) a TLS 1.3 (1-RTT) con RTT = 45 ms) expresado en ms.
4 [PRB-0030] (Avanzado, Diseño) [CH-05.4] Calcule (Hilos concurrentes máximos de Argon2id (64 MiB) en un servidor de autenticación con 4096 MiB dedicados) expresado en hilos.

Capítulo 6

Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD

Ingeniería de plataformas, contenedores Linux (namespaces y cgroups v2), orquestación declarativa, infraestructura como código e integración y entrega continua.

Consejo

Declarar la infraestructura en código versionado con estado remoto bloqueable elimina la deriva de configuración (Configuration Drift) entre entornos de pruebas y producción.

Definición

Anatomía de los Contenedores Linux: Namespaces, cgroups v2 y OverlayFS: Un contenedor no es una máquina virtual con hipervisor propio, sino un grupo de procesos aislados en el mismo kernel Linux mediante namespaces y acotados en recursos mediante cgroups v2.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Desacoplar el despliegue técnico (Deployment) del lanzamiento funcional (Release) mediante despliegues Canary y Feature Flags reduce el radio de impacto de regresiones en producción.

6.1 Anatomía de los Contenedores Linux: Namespaces, cgroups v2 y OverlayFS y Orquestación con Kubernetes: Bucle de Reconciliación, Pods, Probes y Autoscaling

El kernel Linux proporciona tres mecanismos que sustentan el estándar OCI (Open Container Initiative): 1) **Namespaces** (`pid`, `net`, `mnt`, `uts`, `ipc`, `user`, `cgroup`), que restringen qué procesos, interfaces de red y puntos de montaje puede observar el contenedor; 2) **Control Groups (cgroups v2)**, que contabilizan y limitan el consumo de CPU, memoria RAM y E/S de disco; y 3) sistemas de archivos en unión (**OverlayFS**), que superponen capas inmutables direccionadas por hash SHA-256 con una capa superior efímera de escritura.

```
# Etapa 1: Compilación estática reproducible
FROM rust:1.80-alpine AS builder
WORKDIR /src
COPY Cargo.toml Cargo.lock ./
COPY src ./src
RUN cargo build --release --locked

# Etapa 2: Imagen mínima sin shell ni usuario root
FROM gcr.io/distroless/cc-debian12:nonroot
COPY --from=builder /src/target/release/motor-kv /usr/local/bin/motor-kv
USER nonroot:nonroot
ENTRYPOINT ["/usr/local/bin/motor-kv"]
```

En Kubernetes, la unidad mínima desplegable es el **Pod** (uno o más contenedores que comparten namespace de red y volúmenes). Los controladores (`Deployment`, `StatefulSet`, `DaemonSet`) supervisan la salud de los pods mediante tres sondas: `startupProbe`, `readinessProbe` (que retira el pod del balanceador `Service` cuando no puede recibir tráfico sin matarlo) y `livenessProbe` (que reinicia el contenedor

ante interbloqueos fatales). Todo contenedor de producción debe declarar `resources.requests` (usados por el `kube-scheduler`) y `limits`.

6.2 Infraestructura como Código (IaC) con Terraform y Gestión de Estado y Pipelines CI/CD y Estrategias de Despliegue Progresivo (Blue/Green, Canary, Feature Flags)

Terraform construye un grafo acíclico dirigido (DAG) de dependencias entre recursos cloud (redes VPC, clústeres EKS/GKE, bases de datos RDS, políticas IAM) y calcula un plan de ejecución determinista (`terraform plan`) antes de aplicar mutaciones (`terraform apply`). El archivo de estado (`terraform.tfstate`) debe almacenarse en un backend remoto cifrado con bloqueo transaccional y jamás incluir secretos en texto plano. Un pipeline de Integración Continua (CI) verifica automáticamente en cada Pull Request: formateo, análisis estático (linters y chequeo de tipos), pruebas unitarias y de integración, auditoría de vulnerabilidades (CVE) y compilación del artefacto firmado. En Entrega Continua (CD), un despliegue **Canary** encamina inicialmente el $1-5\,\%$ del tráfico real a la nueva versión y promueve automáticamente al $100\,\%$ sólo si los SLIs de tasa de error y latencia p99 permanecen dentro del umbral.

Problema resuelto

Enunciado Un Deployment de Kubernetes tiene actualmente 4 pods en ejecución con una utilización media de CPU del $90\,\%$. Si el Horizontal Pod Autoscaler (HPA) está configurado para mantener una utilización objetivo del $60\,\%$ según la fórmula $R_des = R_act*(U_act/U_obj)$, calcule el número de réplicas al que escalará el clúster.

Datos

- $R_{act} = 4\,\text{pods}$ — Réplicas actuales en ejecución
- $U_{act} = 90\ \%$ — Utilización media actual de CPU
- $U_{obj} = 60\ \%$ — Utilización objetivo configurada en el HPA

Incógnita Número deseado de réplicas R_des en pods

Modelo físico

Dimensionamiento de réplicas Kubernetes mediante Horizontal Pod Autoscaler (HPA)

$$R_{\text{des}} = \frac{4 \cdot 90}{60}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 4 réplicas actuales por la razón entre la utilización actual ($90\,\%$) y la objetivo ($60\,\%$)

$$R_{\text{des}} = \frac{4 \cdot 90}{60}$$

$$\frac{(4 \cdot 90)}{60}$$

Resultado

$$Resultado = 6$$

Interpretación El controlador HPA escala el Deployment a 6 pods para restablecer la utilización media de CPU en el $60\,\%$.

Comprobación de los pasos

- Paso 1: 6 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) [CH-06.1] Calcule (Reducción en MB de una imagen Docker al usar multi-stage distroless de 22 MB frente a la imagen de compilación de 840 MB) expresado en MB.
2 [PRB-0034] (Intermedio, Desarrollo por pasos) [CH-06.2] Calcule (Solicitudes de CPU totales en millicores reservadas por un Deployment de 6 pods con request de 250m cada uno) expresado en mCPU.
3 [PRB-0035] (Intermedio, Desarrollo por pasos) [CH-06.3] Calcule (Tráfico en req/s recibido por la versión Canary (5 %) en un servicio que recibe 12 000 req/s totales) expresado en req/s.
4 [PRB-0036] (Avanzado, Diseño) [CH-06.4] Calcule (Pods disponibles mínimos durante un RollingUpdate de 10 réplicas con maxUnavailable = 2) expresado en pods.

Capítulo 7

Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados

Síntesis de las 19 rutas formativas de la enciclopedia y análisis técnico de los 6 proyectos integradores verificados incluidos en el repositorio.

Consejo

Los proyectos de lenguajes dinámicos, tipado estructural y bases de datos demuestran compilación a bytecode, control de flujo reactivo con Token Bucket y contabilidad de doble partida en BCNF.

Definición

Arquitectura de los 19 Itinerarios de Aprendizaje Personalizados: El grafo acíclico dirigido (DAG) de prerrequisitos permite trazar rutas óptimas tanto para quien comienza desde cero absoluto como para ingenieros que se especializan en un nuevo dominio.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La evaluación de competencias combina corrección algorítmica, seguridad de memoria y tipos, claridad arquitectónica, cobertura de pruebas y observabilidad en producción.

7.1 Arquitectura de los 19 Itinerarios de Aprendizaje Personalizados y Proyectos Integradores de Sistemas: Motor KV en Rust, Arena Allocator en C17 y Planificador en C++20

La enciclopedia estructura 19 itinerarios completos documentados en `blueprint/learning-paths.json`: 1) **Cero Absoluto a Primer Empleo Profesional**, 2) **Grado Universitario Completo en Ciencias de la Computación (ACM/IEEE CS2023)**, 3) **Ingeniería de Sistemas y Bajo Nivel (C, C++, Rust, Zig, ASM)**, 4) **Desarrollo Web Full-Stack Moderno (HTML, CSS, TS, React/Next.js, SQL)**, 5) **Ingeniería Backend y Sistemas Distribuidos (Go, Java/Spring, C#/.NET, Rust/Axum)**, 6) **Ciencia de Datos, Computación Científica e IA (Python, R, Julia, SQL)**, 7) **Desarrollo Móvil Nativo y Multiplataforma (Swift, Kotlin, Dart/Flutter)**, 8) **Ingeniería de Bases de Datos y Plataformas de Datos**, 9) **Cloud, DevOps, SRE y Ciberseguridad**, 10) **Programación Funcional y Lenguajes Declarativos (Haskell, OCaml, Elixir, Erlang, Scala, F#, Clojure, Prolog)**, y 9 rutas adicionales de especialización directa por ecosistema. 1) En `projects/rust-kv-engine`, se desarrolla en Rust 2021 un motor de almacenamiento clave-valor persistente basado en Write-Ahead Log (WAL) binario con sumas de verificación CRC32 (polinomio IEEE `0xEDB88320`), compactación atómica y recuperación ante corrupción. 2) En `projects/c-arena-allocator`, se implementa en C17 estricto un asignador lineal de memoria (*Arena Allocator*) con alineación a potencias de dos y un analizador de paquetes binarios de red de copia cero (*zero-copy*) verificado con AddressSanitizer y UndefinedBehaviorSanitizer. 3) En `projects/cpp-task-scheduler`, se construye en C++20 un planificador de tareas por prioridad restringido estáticamente mediante `concept ExecutableTask` y `std::ranges`.

7.2 Proyectos Integradores de Alto Nivel y Datos: VM en Python 3.12, Router HTTP en TypeScript y Ledger SQL y Rúbricas de Evaluación por Niveles de Competencia (Principiante a Experto)

4) En `projects/python-bytecode-vm`, se implementa en Python 3.12 un compilador de expresiones aritméticas y variables a bytecode con plegado de constantes (*Constant Folding*) y su máquina virtual de pila, verificado con `pytest`, `ruff` y `pyright`. 5) En `projects/ts-reactive-http`, se construye en TypeScript 5 un enrutador HTTP declarativo con tubería de middlewares y limitador de tasa (*Token Bucket*) ejecutado nativamente en Node.js. 6) En `projects/sql-transactional-ledger`, se diseña en SQL un libro mayor contable de doble partida en BCNF con triggers de invariantes, CTEs recursivas de jerarquía de cuentas y funciones de ventana. Cada proyecto e itinerario se evalúa mediante cinco dimensiones objetivas: 1) **Exactitud funcional y manejo de casos límite**, 2) **Uso idiomático del lenguaje y de su sistema de tipos**, 3) **Complejidad temporal y espacial asintótica y constantes reales de caché/E/S**, 4) **Calidad de pruebas automatizadas (unitarias, propiedades y sanitizers)**, y 5) **Documentación técnica, reproducibilidad del build y ausencia total de advertencias del compilador**.

Problema resuelto

Enunciado Un estudiante autodidacta aborda el itinerario de Ingeniería Backend y Sistemas Distribuidos, que comprende 720 horas de estudio teórico y proyectos prácticos. Si dedica de forma constante 15 horas semanales, calcule el número exacto de semanas necesarias para completar la especialización.

Datos

- $H_{tot} = 720\,\text{horas}$ — Horas totales del itinerario formativo
- $h_{sem} = 15\,\text{horas}\,\text{semana}^{-1}$ — Dedicación semanal constante

Incógnita Duración del itinerario S_sem en semanas

Modelo físico

Planificación temporal de un itinerario formativo completo de 720 horas

$$S_{\text{sem}} = \frac{720}{15}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos las 720 horas totales del itinerario entre las 15 horas semanales

$$S_{\text{sem}} = \frac{720}{15}$$

$$\frac{720}{15}$$

Resultado

$$\mathit{Resultado} = 48$$

Interpretación Con una dedicación sostenida de 15 horas semanales, el estudiante completa el itinerario profesional en 48 semanas (menos de un año natural).

Comprobación de los pasos

- Paso 1: 48 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 **[PRB-0039] (Básico, Numérico)** [CH-07.1] Calcule (Horas dedicadas a práctica y proyectos si representan el 60 % de un itinerario de 720 horas) expresado en horas.
2 **[PRB-0040] (Intermedio, Desarrollo por pasos)** [CH-07.2] Calcule (Total de pruebas ejecutadas al correr las 6 suites de los proyectos integradores con un promedio de 5 aserciones por proyecto) expresado en aserciones.
3 **[PRB-0041] (Intermedio, Desarrollo por pasos)** [CH-07.3] Calcule (Bytes de cabecera por registro en el motor WAL de Rust (4 B CRC32 + 4 B $\text{key}_l en + 4B$ $\text{val}_l en + 1B$ op)) expresado en bytes.
4 **[PRB-0042] (Avanzado, Diseño)** [CH-07.4] Calcule (Saldo neto verificado en una partida contable doble donde la suma de cargos es 18 500 y la de abonos es 18 500) expresado en unidades.

Apéndice A

Notación

Símbolo	Significado	Unidades
$A = T_{\mathrm{MTTF}}/T_{\mathrm{tot}}$	Disponibilidad estacionaria de un sistema reparable	fracción
$I = \frac{C_e}{C_a + C_e}$	Métrica de inestabilidad de paquetes de Robert C. Martin	adimensional
$N \geq 2f + 1$	Nodos requeridos en consenso tolerante a f fallos de parada (Raft/Paxos)	nodos

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización

Problema resuelto [PRB-0001]
En una arquitectura modular, el paquete de facturación es importado por 18 módulos externos (acoplamiento aferente Ca = 18) e importa únicamente 2 bibliotecas estándar de tipos (acoplamiento eferente Ce = 2). Calcule el índice de inestabilidad I expresado en porcentaje.
Respuesta

$$Resultado = 10$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[CH-01.1] Calcule (Índice de inestabilidad porcentual de un adaptador HTTP con Ca = 1 y Ce = 9) expresado en %.
Respuesta 90
Ejercicio 2 [PRB-0004]
[CH-01.2] Calcule (Distancia a la secuencia principal en porcentaje si A = 40 % e I = 50 % (|40 + 50 - 100|)) expresado en %.
Respuesta 10
Ejercicio 3 [PRB-0005]
[CH-01.3] Calcule (Reducción de complejidad ciclomática al descomponer un método de complejidad 19 en 4 funciones de complejidad 2) expresado en puntos.
Respuesta 11
Ejercicio 4 [PRB-0006]
[CH-01.4] Calcule (Número de clases eliminadas al reemplazar 6 clases Strategy de un solo método por funciones de primera clase) expresado en clases.
Respuesta 6

Cálculos reproducidos

$\frac{(2 \cdot 100)}{(18+2)} = 10$

C.2 Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC)

Problema resuelto [PRB-0007]
Una consulta GraphQL solicita una lista de 50 pedidos junto con los datos del autor de cada pedido. Sin agrupación (N+1), el servidor ejecuta 1 consulta para la lista y 50 consultas individuales de 2 ms de latencia cada una. Con DataLoader, ejecuta solo 2 consultas (la lista y un único SELECT ... WHERE id IN (...)) de 2 ms cada una. Calcule el tiempo total de base de datos ahorrado en ms.
Respuesta

$$Resultado = 98$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]
[CH-02.1] Calcule (Consultas SQL ahorradas al resolver 120 elementos con DataLoader (2 consultas) frente a N+1 (121 consultas)) expresado en consultas.

Respuesta 119

Ejercicio 2 [PRB-0010]

[CH-02.2] Calcule (Latencia acumulada en ms de una cadena síncrona de 6 microservicios con 8 ms por salto) expresado en ms.

Respuesta 48

Ejercicio 3 [PRB-0011]

[CH-02.3] Calcule (Disponibilidad porcentual aproximada (en centésimas x100) de 3 servicios en serie al 99 % cada uno (10000 - 3*100)) expresado en centésimas de %.

Respuesta 9700

Ejercicio 4 [PRB-0012]

[CH-02.4] Calcule (Tamaño de página máximo devuelto si una API REST limita $\text{page}_size <= 100$ cuando el cliente solicita 5000) expresado en registros.

Respuesta 100

Cálculos reproducidos

$(((50 + 1) \cdot 2) - (2 \cdot 2)) = 98$

C.3 Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing

Problema resuelto [PRB-0013]

Una herramienta de Mutation Testing genera 250 mutantes sintácticos sobre el módulo de liquidación financiera. De ellos, 10 son mutantes equivalentes (no alteran la semántica) y la batería de pruebas detecta y hace fallar 216 de los mutantes restantes. Calcule el Mutation Score porcentual sobre los mutantes no equivalentes.

Respuesta

$$Resultado = 90$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]

[CH-03.1] Calcule (Mutantes supervivientes que requieren nuevos tests en el módulo auditado (250 - 10 - 216)) expresado en mutantes.

Respuesta 24

Ejercicio 2 [PRB-0016]

[CH-03.2] Calcule (Objetos nuevos creados en Git al modificar 1 archivo dentro de un subdirectorio de profundidad 2 (1 blob + 3 trees + 1 commit)) expresado en objetos.

Respuesta 5

Ejercicio 3 [PRB-0017]

[CH-03.3] Calcule (Casos generados en 5 segundos por un fuzzer libFuzzer que ejecuta 18 000 iteraciones por segundo) expresado en ejecuciones.

Respuesta 90000

Ejercicio 4 [PRB-0018]

[CH-03.4] Calcule (Tiempo total de CI en segundos para 1 200 tests unitarios de 2 ms y 30 tests de integración de 200 ms) expresado en s.

Respuesta 8.4

Cálculos reproducidos

$\frac{(216 \cdot 100)}{(250 - 10)} = 90$

C.4 Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE

Problema resuelto [PRB-0019]
Un servicio crítico de pagos define un SLO de disponibilidad del 99,9 % medido sobre una ventana de 30 días (43 200 minutos totales). Calcule el presupuesto de error mensual en minutos durante los cuales el servicio puede experimentar indisponibilidad sin violar su SLO.
Respuesta

$$Resultado = 43{,}2$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
[CH-04.1] Calcule (Presupuesto de error anual en minutos para un SLO de cuatro nueves 99,99 % (525600 / 10000)) expresado en min.
Respuesta 52.56
Ejercicio 2 [PRB-0022]
[CH-04.2] Calcule (Nodos totales mínimos en un clúster Raft para tolerar f = 3 fallos simultáneos (2 * 3 + 1)) expresado en nodos.
Respuesta 7
Ejercicio 3 [PRB-0023]
[CH-04.3] Calcule (Tope de espera en ms en el cuarto reintento (intento=4) con backoff exponencial de base 50 ms ($50 * 2^4$)) expresado en ms.
Respuesta 800
Ejercicio 4 [PRB-0024]
[CH-04.4] Calcule (Disponibilidad porcentual de un servicio con MTTF = 990 horas y MTTR = 10 horas (990 * 100 / (990 + 10))) expresado en %.
Respuesta 99

Cálculos reproducidos

$\frac{43200}{1000} = 43{,}2$

C.5 Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain

Problema resuelto [PRB-0025]
Un servidor configura Argon2id con un coste de memoria de 64 MiB por verificación de contraseña y 3 iteraciones. Si un atacante intenta evaluar en paralelo 512 candidatos simultáneos en una GPU, calcule la memoria VRAM total en GiB (1 GiB = 1024 MiB) necesaria para sostener esos 512 hilos sin penalización de ancho de banda.
Respuesta

$$Resultado = 32$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
[CH-05.1] Calcule (Bytes totales de sobrecoste criptográfico en AES-256-GCM por mensaje (12 bytes de nonce IV + 16 bytes de tag)) expresado en bytes.
Respuesta 28
Ejercicio 2 [PRB-0028]
[CH-05.2] Calcule (Bits de entropía de una clave aleatoria de 32 bytes generada con un CSPRNG del sistema operativo (32 * 8)) expresado en bits.
Respuesta 256
Ejercicio 3 [PRB-0029]

[CH-05.3] Calcule (Reducción de latencia de establecimiento en ms al pasar de TLS 1.2 (2-RTT) a TLS 1.3 (1-RTT) con RTT = 45 ms) expresado en ms.
Respuesta 45
Ejercicio 4 [PRB-0030]
[CH-05.4] Calcule (Hilos concurrentes máximos de Argon2id (64 MiB) en un servidor de autenticación con 4096 MiB dedicados) expresado en hilos.
Respuesta 64

Cálculos reproducidos

$\frac{(512 \cdot 64)}{1024} = 32$

C.6 Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD

Problema resuelto [PRB-0031]
Un Deployment de Kubernetes tiene actualmente 4 pods en ejecución con una utilización media de CPU del 90 %. Si el Horizontal Pod Autoscaler (HPA) está configurado para mantener una utilización objetivo del 60 % según la fórmula $R_des = R_act * (U_act/U_obj)$, calcule el número de réplicas al que escalará el clúster.
Respuesta

$$Resultado = 6$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0033]
[CH-06.1] Calcule (Reducción en MB de una imagen Docker al usar multi-stage distroless de 22 MB frente a la imagen de compilación de 840 MB) expresado en MB.
Respuesta 818
Ejercicio 2 [PRB-0034]
[CH-06.2] Calcule (Solicitudes de CPU totales en millicores reservadas por un Deployment de 6 pods con request de 250m cada uno) expresado en mCPU.
Respuesta 1500
Ejercicio 3 [PRB-0035]
[CH-06.3] Calcule (Tráfico en req/s recibido por la versión Canary (5 %) en un servicio que recibe 12 000 req/s totales) expresado en req/s.
Respuesta 600
Ejercicio 4 [PRB-0036]
[CH-06.4] Calcule (Pods disponibles mínimos durante un RollingUpdate de 10 réplicas con maxUnavailable = 2) expresado en pods.
Respuesta 8

Cálculos reproducidos

$\frac{(4 \cdot 90)}{60} = 6$

C.7 Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados

Problema resuelto [PRB-0037]
Un estudiante autodidacta aborda el itinerario de Ingeniería Backend y Sistemas Distribuidos, que comprende 720 horas de estudio teórico y proyectos prácticos. Si dedica de forma constante 15 horas semanales, calcule el número exacto de semanas necesarias para completar la especialización.
Respuesta

$$Resultado = 48$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0039]

[CH-07.1] Calcule (Horas dedicadas a práctica y proyectos si representan el 60 % de un itinerario de 720 horas) expresado en horas.

Respuesta 432

Ejercicio 2 [PRB-0040]

[CH-07.2] Calcule (Total de pruebas ejecutadas al correr las 6 suites de los proyectos integradores con un promedio de 5 aserciones por proyecto) expresado en aserciones.

Respuesta 30

Ejercicio 3 [PRB-0041]

[CH-07.3] Calcule (Bytes de cabecera por registro en el motor WAL de Rust (4 B CRC32 + 4 B key$_l$$en$+4$B$ val$_l$$en$ + 1B op)) expresado en bytes.

Respuesta 13

Ejercicio 4 [PRB-0042]

[CH-07.4] Calcule (Saldo neto verificado en una partida contable doble donde la suma de cargos es 18 500 y la de abonos es 18 500) expresado en unidades.

Respuesta 0

Cálculos reproducidos

$\frac{720}{15} = 48$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 28 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 2 valor o condición.
¶21: dos testimonios. Ecuaciones: 1 valor o condición.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.

Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0) & Clean Architecture (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
In Search of an Understandable Consensus Algorithm (Raft) & Site Reliability Engineering (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10,

¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
OWASP Application Security Verification Standard (ASVS 4.0) & NIST SP 800-63B (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
REV-0001: ch-01 scientific
REV-0002: ch-02 scientific
REV-0003: ch-03 scientific
REV-0004: ch-04 scientific
REV-0005: ch-05 scientific
REV-0006: ch-06 scientific
REV-0007: ch-07 scientific

Apéndice E

Plan de estudio

Partes IX–XIII: Ingeniería Profesional, Cloud, Seguridad e Itinerarios

✓Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización

- Evaluar cuantitativamente el acoplamiento aferente y eferente, aplicar los principios SOLID sin sobreingeniería e implementar patrones de diseño GoF con tipos modernos.

2 conceptos planificados.

✓Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC)

- Diseñar sistemas bajo Arquitectura Hexagonal y Domain-Driven Design, elegir justificadamente entre monolitos modulares y microservicios, y especificar contratos de API REST, GraphQL y gRPC.

2 conceptos planificados.

✓Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing

- Comprender el almacén direccionable por contenido de Git, diseñar estrategias de pruebas en pirámide/trofeo y verificar invariantes con Property-Based Testing, Fuzzing y Mutation Testing.

2 conceptos planificados.

✓Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE

- Razonar sobre tiempo lógico y orden parcial, implementar patrones de resiliencia (Circuit Breaker, Backoff con Jitter, Idempotencia), instrumentar sistemas con OpenTelemetry y gestionar presupuestos de error SRE.

2 conceptos planificados.

✓Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain

- Mitigar vulnerabilidades del OWASP Top 10, aplicar primitivas criptográficas estándar (Argon2id, AES-GCM, Ed25519), diseñar flujos OAuth 2.1 / OIDC con PKCE y asegurar la cadena de suministro de software.

2 conceptos planificados.

✓Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD

- Construir imágenes OCI multi-stage mínimas y sin privilegios root, operar clústeres Kubernetes declarativos, aprovisionar infraestructura con Terraform y automatizar despliegues progresivos.

2 conceptos planificados.

✓Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados

- Planificar un itinerario de estudio personalizado desde nivel cero hasta nivel experto y ejecutar proyectos integradores completos con arquitectura de producción.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0) & Clean Architecture. https://www.computer.org/education/bodies-of-knowledge/software-engineering/vol-ref-1

[27] In Search of an Understandable Consensus Algorithm (Raft) & Site Reliability Engineering. https://raft.github.io/raft.pdf/vol-ref-2

[28] OWASP Application Security Verification Standard (ASVS 4.0) & NIST SP 800-63B. https://owasp.org/www-project-application-security-verification-standard/vol-ref-3